

CHAPTER THREE

RESEARCH METHODOLOGY

3.1 Introduction

This chapter presents the methodology adopted for the design, development and evaluation of the personality prediction system. The system was developed as a web-based research platform for predicting the five Big Five personality traits from English-language online comments. The traits considered are Openness, Conscientiousness, Extraversion, Agreeableness and Neuroticism, represented as OCEAN. The main experiment uses personality-labelled comments from the PANDORA dataset and compares two prediction approaches within a common processing framework.

The implementation combines data preparation, Q-learning-based comment selection, contextual representation using BERT, training-data augmentation using a generative adversarial network and personality prediction using Lasso or ElasticNet regression and a stacked bidirectional Long Short-Term Memory model. The regression model receives a pooled representation of an author's comments, while the LSTM receives a sequence of comment embeddings. Both models produce continuous OCEAN scores. Low and High labels are subsequently obtained through a separate threshold procedure for classification evaluation.

The methodology emphasizes the separation of model fitting, model development decisions and final evaluation. Training authors supply the observations used to fit the selector, augmentation model and prediction models. Validation authors support the selection of LSTM weights and decision thresholds. Test authors provide the held-out observations used for final performance reporting. Eight experimental conditions are formed by combining the two prediction models, two selection approaches and two augmentation settings.

The chapter therefore explains the research design, system analysis, overall architecture, data preparation, prediction pipeline, database structure and evaluation procedure. It also describes the limits of the implementation so that the research experiment is distinguished from the supplementary questionnaire, X data collection and public demonstration functions retained in the application. The procedures are organized around the six objectives established in Chapter One.

3.2 Research Methodology

The development of the personality prediction platform follows a Design and Development Research approach. This approach is appropriate because the study involves the construction of a working software artifact and an examination of its ability to address the identified research problem. The artifact provides an environment in which data can be prepared, models can be trained under defined conditions and the resulting predictions can be examined through a researcher interface.

The development approach is supported by a controlled comparative experiment. System development addresses whether the platform can perform the required operations, while the experiment addresses how the prediction methods behave under common data conditions. A functioning interface alone does not establish prediction quality. Similarly, a model score without a clear record of the data and procedure does not provide a sufficient account of the developed system. Both aspects are therefore considered within the methodology.

The development process consists of requirements analysis, system design, implementation, testing and evaluation. These activities guide the construction of the application and provide a basis for checking whether the completed workflow corresponds to the objectives of the study. The changes introduced during development are reflected in the experimental design rather than treated as unrelated additions to the original pipeline.

3.2.1 Requirements Analysis and Development Procedure

Requirements analysis identifies the information required by the researcher and the operations needed to produce a reviewable personality prediction result. The principal requirements include loading labelled comments, retaining author identities, maintaining separate data partitions, preparing contextual features and comparing prediction conditions. The system must also retain the settings and evaluation records associated with each experiment so that results can be interpreted after training has ended.

System design establishes the relationship between the web interface, data preparation services, machine learning services and persistent records. The processing stages are separated into modules with defined inputs and outputs. For example, the BERT service receives cleaned comments and returns numerical vectors, while the prediction services

receive these vectors with the corresponding training labels. This arrangement allows the same text representations to support more than one model.

Implementation uses Django for the application workflow and Python services for the experimental pipeline. The services perform comment preparation, selection, encoding, augmentation, model fitting and evaluation. The researcher interface provides access to experiment creation, processing status, stored comparisons and individual prediction profiles. Long-running experiment work is separated from the initial web request so that the researcher can review the status of a run after submitting it.

Testing is concerned with the correctness of these operations and their integration. Evaluation is concerned with the quality of the predictions and the effects of the experimental factors. The distinction is maintained throughout the study: a successful data import is a functional outcome, while a lower held-out MAE is a predictive outcome. Neither is used as a replacement for the other.

3.2.2 Experimental Design

The experiment adopts a two-by-two-by-two factorial arrangement. The first factor is prediction model, with sparse regression and LSTM as its two levels. The second factor is comment selection, with full-history baseline selection and Q-learning selection as its two levels. The third factor is augmentation, with no GAN and GAN augmentation as its two levels. Their combination produces the eight conditions shown in Table 3.1.

No.	Prediction model	Comment selection	Augmentation
1	Lasso or ElasticNet	Full-history baseline	No GAN
2	Lasso or ElasticNet	Q-learning	No GAN
3	Lasso or ElasticNet	Full-history baseline	GAN
4	Lasso or ElasticNet	Q-learning	GAN
5	LSTM	Full-history baseline	No GAN
6	LSTM	Q-learning	No GAN
7	LSTM	Full-history baseline	GAN
8	LSTM	Q-learning	GAN

Table 3.1: Eight Experimental Conditions

The eight conditions represent eight combinations of processing choices, rather than eight folds of cross-validation. The same author-level training, validation and test allocations are used across the conditions within a run. This allows a difference between two matched conditions to be examined without introducing a different set of evaluation authors. References to eight-condition training in this chapter therefore describe the factorial experiment.

The sparse regression branch is labelled Lasso in parts of the application, but the current default regularization setting is ElasticNet. Lasso remains a supported alternative within that branch. These alternatives do not add extra rows to the eight-condition design. The actual regularization setting must accompany the result so that a stored Lasso condition is not incorrectly interpreted as a pure L1 model when ElasticNet was used.

The arrangement supports three direct comparisons. The contribution of Q-learning is assessed while model type and augmentation are held constant. The contribution of GAN augmentation is assessed while model type and selection are held constant. The model comparison holds selection and augmentation constant while changing the prediction model. These comparisons provide a more useful explanation of component behaviour than selecting one favourable result from unrelated configurations.

3.2.3 Alignment with the Research Objectives

The first objective is addressed through dataset loading, cleaning, author grouping and the preservation of the three supplied data partitions. The associated evidence consists of the source-file information, retained author counts, available comment counts and recorded partition membership. These records establish which observations were available for model fitting and which were reserved for evaluation.

The second objective is addressed by training a Q-learning selector on training comments and comparing its selections with the full-history baseline. The third objective is addressed by extracting 768-dimensional BERT vectors and retaining both their author-level means and their ordered sequences. Selection counts and feature dimensions provide operational checks for these stages, while matched model results establish whether the representations support useful prediction.

The fourth objective is addressed through the generation of paired synthetic embeddings and OCEAN-score vectors from training observations. The augmentation report records whether underrepresented trait bands were targeted and whether synthetic observations were actually produced. The fifth objective is addressed by fitting the sparse regression and stacked bidirectional LSTM branches under the same experimental arrangement.

The sixth objective is addressed through held-out evaluation of all eight conditions. Continuous-score measures, Low/High measures, threshold records and comment-selection efficiency are retained together. This connection between objectives, procedures and evidence prevents the chapter from describing functions that have no role in the main research question. It also provides a basis for organizing the experimental results in the subsequent chapter.

3.3 System Analysis

System analysis identifies the main components of the platform and the movement of information between them. The current research workflow begins with an authenticated researcher and a prepared source of PANDORA data. It proceeds through experiment configuration, data preparation, model execution and result review. The platform retains earlier participant-management functions, but new volunteer registration is not a prerequisite for the PANDORA experiment.

The main input is an author's collection of comments with an associated five-score personality label. The main output is a set of predicted trait scores and evaluation records identified by experimental condition. The author is the unit of prediction. Individual comments provide the evidence from which the author's representation is formed; they are not counted as separate people when model performance is calculated.

The principal functional modules are researcher management, dataset preparation, experiment execution and reporting. Their separation supports changes to an individual processing stage without requiring the entire application to be redesigned. At the same time, the run identifier links their outputs so that the researcher can follow an experiment from its submitted settings to its stored results.

3.3.1 Researcher Management Module

The researcher management module provides registration, login and account-related functions. Experiment and prediction-run records are associated with researcher accounts. The interface uses authenticated views for research tools, enabling the application to organize experiment history and prediction records around the researcher who initiated the operation.

Researcher management also provides an operational boundary between the research tools and the public demonstration page. The researcher tools expose experiment settings and stored evaluation evidence, whereas the public page gives a simplified demonstration of personality-related output. An account record establishes ownership within the application; it does not, by itself, establish that every possible deployment has undergone a complete security assessment.

3.3.2 Dataset Preparation Module

The dataset preparation module loads the configured training, validation and test files. It recognizes the comment, author and OCEAN fields, groups comments under their authors and applies the shared cleaning procedure. Eligible authors are then selected according to the configured sample size and minimum usable-comment requirement.

The module retains enough information to connect prepared data with their source files and experimental partitions. Prepared-data caching reduces the need to repeat cleaning during development. When the source data or cleaning procedure changes, the prepared cache must be refreshed so that a later experiment does not silently use an earlier representation of the dataset.

3.3.3 Personality Prediction and Experiment Module

The experiment module coordinates the eight conditions. It trains the comment selector on training authors, constructs baseline and Q-learning features, applies augmentation where required and fits the two prediction branches. The same feature collections are reused by conditions that have the same selection setting, reducing unnecessary repetition during the comparison.

The module also coordinates validation and test prediction. It records continuous outputs, derives binary labels through the defined threshold procedure and sends the results

to a common metrics service. This organization helps prevent the different model branches from using unrelated definitions of accuracy, F1-score or prediction error.

3.3.4 Reporting and Visualization Module

The reporting module presents experiment history, condition comparisons, threshold results and prediction profiles. Researchers can examine which model and selection method produced a result, whether augmentation was requested and how the condition performed on the evaluated data. Supporting records include data allocation, quality summaries and generated output files.

The module also supports graphical summaries of model performance and the five personality dimensions. These displays make stored results easier to compare, but their meaning depends on the source of the values. A profile chart shows estimated trait scores for an author, while a comparison chart summarizes performance across authors. The two types of display answer different questions and are interpreted separately.

3.4 Overall System Architecture

The platform adopts a modular architecture that separates presentation, application coordination, machine learning and data persistence. Django connects these parts through the researcher interface and service functions. The architecture supports the complete experimental workflow while retaining earlier application features for participant records, questionnaires and X data acquisition.

The presentation layer contains the research dashboard, training form, experiment history, condition detail pages and prediction profile pages. The application layer validates submitted settings, creates run records and initiates processing. The machine learning layer performs preparation, selection, BERT encoding, augmentation, model fitting and evaluation. The persistence layer retains database records and saved experiment artifacts.

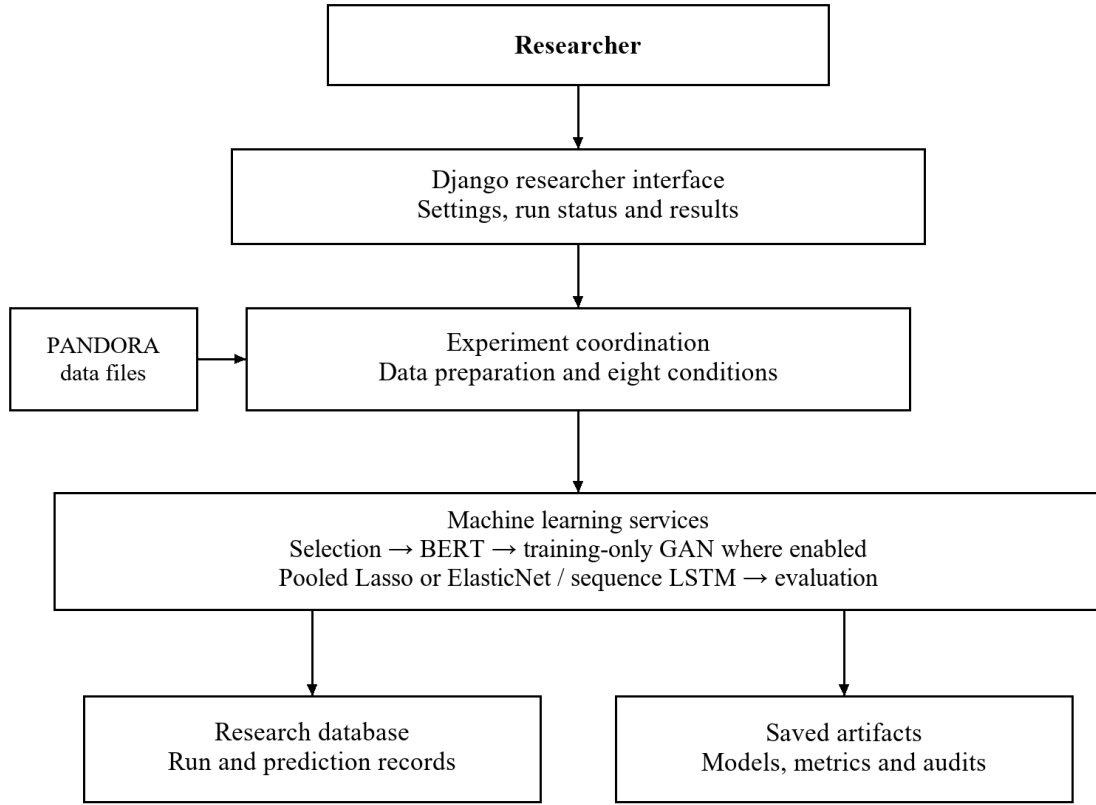


Figure 3.1: High-Level Architecture of the Personality Prediction System

Figure 3.1 shows the PANDORA experiment as the analytical path used by the research platform. The diagram separates source data from recorded experiment results so that the database is not mistaken for the original personality dataset. It also shows the model branches as alternatives within the experiment. The LSTM is not a final stage that every Lasso prediction must pass through.

3.4.1 Application Coordination and Processing

When a researcher submits a training request, the application creates an experiment record containing the requested settings. The current PANDORA interface starts the experiment in a background Python thread. The processing function updates the run as it proceeds and records completion or an error. This allows the web request to return while the researcher later checks the run status.

Celery support is also present in the application and is used by parts of the earlier volunteer pipeline. It is therefore necessary to distinguish the two execution paths. The PANDORA dashboard experiment is not described as a Celery-managed task merely because

Celery is configured elsewhere in the project. A background thread supports the present workflow, but it does not provide the same independent worker persistence that a separate task queue would provide.

The training services can also be called through the experiment runner outside the web interface. Both forms of execution use the same experimental components. The report identifies the configuration actually used for a run rather than assuming that every entry point has identical defaults or dataset handling. This is particularly important where older convenience functions remain available for development.

3.4.2 Storage and Reproducibility

The database stores run summaries, condition results, threshold rows, dataset allocations and prediction profiles. Larger experimental outputs, including fitted model files and detailed comparison files, are retained in an artifact directory associated with the run. The database record therefore provides the connection between the researcher-facing result and the files needed for a more detailed review.

Reproducibility information includes the random seed, configuration, source files, author allocation and repository identity where available. A seed helps repeat sampling and stochastic model operations, but it is only one part of the record. Changes to the dataset, preprocessing, library environment or model configuration can change the result even when the same seed is retained.

This arrangement also separates stored results from new predictions. A completed experiment can be reviewed without fitting its models again. When a further prediction batch is requested, its own record identifies the source experiment and selected condition. This prevents a later profile from losing the context of the model from which it was produced.

3.5 Data Acquisition and Preparation

The quality of personality prediction depends on the quality and organization of the observations supplied to the models. The main experiment uses an existing personality-labelled dataset rather than collecting all training labels through the application's questionnaire interface. PANDORA provides Reddit comments associated with personality information, including Big Five labels (Gjurković et al., 2021). The present study uses the available Big Five comment data prepared for the project.

The local dataset files are supplied as separate training, validation and test collections. These project files are the immediate inputs to the experiment. Their names and partitions should not be interpreted as a claim that the original PANDORA publication prescribes the exact local allocation used in this application. Dataset provenance and local preparation are related but distinct parts of the research record.

3.5.1 Dataset Loading and Author Grouping

The preferred loading path reads Training dataset.xlsx, Validation dataset.xlsx and Test dataset.xlsx from the configured dataset location. The loader also supports relevant parquet files where they are available. Field names are normalized so that common representations of comment text, author identity and the five personality traits can be interpreted consistently.

Comments are grouped using the author field. Each prepared author record contains an identifier, a collection of cleaned comments and an OCEAN-score vector. This grouping is necessary because the study estimates an author's personality from several pieces of text. Dividing the comments randomly without considering authors could place the same person's language in both fitting and evaluation data.

Earlier exports without author identifiers used a combination of trait values as a proxy grouping key. Such a combination is not a reliable substitute for an author identity because different people may share the same scores. The current file-defined path requests author grouping. The loader still contains a fallback for missing authors, so the presence of the author field remains an input requirement that must be checked before a run is accepted as an author-level experiment.

Labels are taken from the supplied personality data. They are not recomputed from the comments, generated by BERT or obtained from a newly administered BFI-44 survey for this experiment. Keeping the source labels separate from the predicted scores makes it possible to compare the model output with an independently supplied target.

3.5.2 Data Validation and Eligibility

The preparation process checks for missing text and records that cannot supply the expected personality information. Empty comments do not proceed as useful text. Authors without the required trait information or without enough usable comments are excluded from

sampling. The current experiment configuration uses a minimum of five cleaned comments per eligible author unless the configuration is changed.

Eligibility is checked after cleaning because a raw comment count may include duplicates, empty entries or text removed by the cleaning rules. For example, an author with several imported rows may have fewer usable comments once repeated text is removed. The retained count is therefore more relevant to feature construction than the number of rows originally read from a file.

The available data may contain more eligible authors than requested for a run. In that case, the sampler uses the configured random seed to choose a subset. Author identifiers are sorted to make the sampling procedure stable for the same input. Where fewer eligible authors are available, the actual retained count is recorded rather than assuming that the requested sample size was achieved.

Input review also considers whether all five labels are numeric and whether a consistent label scale is used across the files. The preparation code provides filtering and reporting, but a successful file load is not treated as proof that every data-quality issue has been resolved. Missing author fields, inconsistent score conventions and overlapping author identities require examination before the corresponding results are used as final evidence.

3.5.3 Text Preprocessing

The shared cleaning service transforms raw comment text into a consistent representation. It removes web links, normalizes whitespace and converts the text to lowercase. Repeated content and text below the minimum length are filtered according to the cleaning rules. The current cleaner uses a minimum character length of three, while the separate author eligibility check controls the minimum number of usable comments.

The cleaning service retains the original text alongside the cleaned text and can preserve extracted structural information such as mentions, hashtags and links. However, the PANDORA selector operates on cleaned comment strings. It does not use the engagement counts or publication recency that appeared in the earlier X-based selection design. Metadata retained by the general application must therefore not be confused with features used by the main experiment.

The preprocessing stage preserves ordinary wording needed for contextual language representation. It does not reduce the experiment to a list of manually selected personality keywords. BERT tokenization occurs later, using the tokenizer associated with the selected pretrained model. This keeps deterministic text cleaning separate from the model-specific conversion of words into input tokens.

Prepared comments retain their order from the loaded and cleaned data. The sequence branch uses this order when constructing the author representation. Where verified timestamps are unavailable, the sequence is described as the retained comment order rather than a confirmed chronological history. This distinction limits the interpretation of the LSTM to patterns across the supplied sequence.

3.5.4 Training Validation and Test Allocation

The three file-defined partitions are prepared separately and combined with explicit index ranges for training, validation and test authors. The training portion is used for fitting the Q-learning policy, the GAN and the prediction models. The validation portion is used for development decisions. The test portion is held aside for the final comparison of fitted conditions.

The sample-size setting in the file-defined loader refers to the requested training-author count. Validation and test requests are derived separately using the configured ratios. For example, the default request of 40 training authors with validation and test ratios of 0.2 requests eight authors from each of the other files, subject to eligibility. This is a configuration example, not a statement of the number of authors used in a completed experiment.

Author membership must be checked across all three partitions. The loader maintains file membership, while the research-contract checks examine whether the recorded author sets overlap. These audit checks report problems; their presence does not guarantee that every invalid run is automatically prevented. Only runs with suitable, distinct author partitions support the intended independent test evaluation.

An older runner path accepts a single prepared list and creates a training and validation split. That path reuses the validation indices for its test field. It is not equivalent to the preferred three-file procedure and must not be reported as an untouched test experiment.

The present methodology uses the file-defined route in accordance with the first objective in Chapter One.

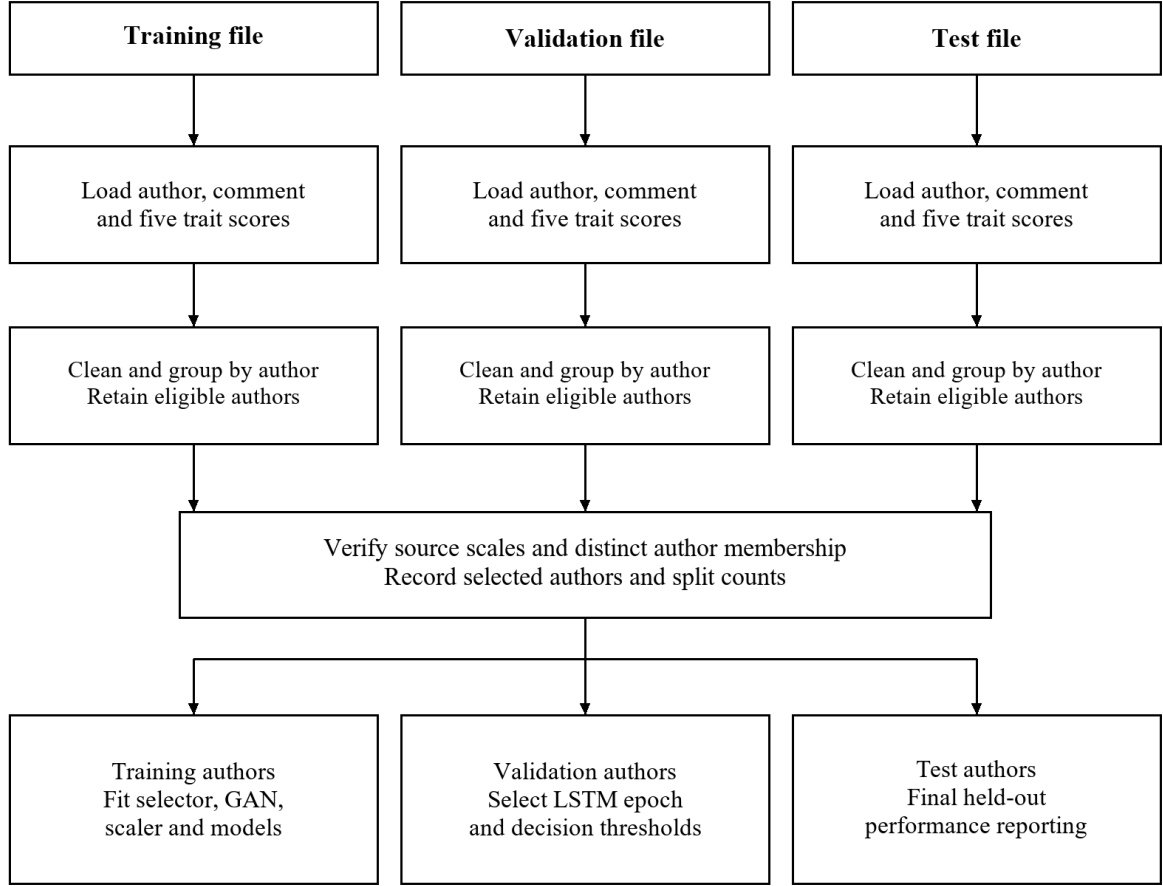


Figure 3.2: Data Preparation and Author-Level Partitioning

3.5.5 Label Scaling and Trait Distribution

The model targets are represented on a normalized scale. Where the source values are on a 0 to 100 scale, division by 100 converts them to the unit interval. A supported 1 to 5 scale is converted by subtracting one and dividing by four. Values already on the unit scale are retained. These fixed transformations make the target values suitable for a common comparison across the prediction branches.

The loader can also fall back to observed-range scaling for an unfamiliar label range. Such a fallback requires review because its boundaries are estimated from the supplied values and may differ between partitions. The main experiment should use the verified common source scale. A warning about incompatible scales cannot be ignored simply because the model fitting stage completes successfully.

Two forms of trait grouping serve different purposes. Training medians define the true Low/High label boundary separately for each trait. Fixed bands at one-third and two-thirds of the normalized scale are used for the imbalance and targeted augmentation procedures. These bands identify sparsely represented score regions; the Medium band does not become a third class in the main binary evaluation.

Training weights reflect the frequency of these trait bands. The implementation calculates inverse-frequency contributions for each trait, averages them for each training author and normalizes the resulting weights. Authors in less represented regions can therefore contribute more strongly to fitting. The weights are based on training labels and are reused across matched conditions, allowing augmentation to be compared against a common weighting procedure.

3.5.6 Supplementary Data Acquisition Functions

The application retains volunteer registration, CSV import of BFI-44 responses, questionnaire scoring and X content retrieval. These functions support the earlier participant-oriented workflow. They are useful application capabilities, but they do not define the source labels or sample allocation of the main PANDORA comparison.

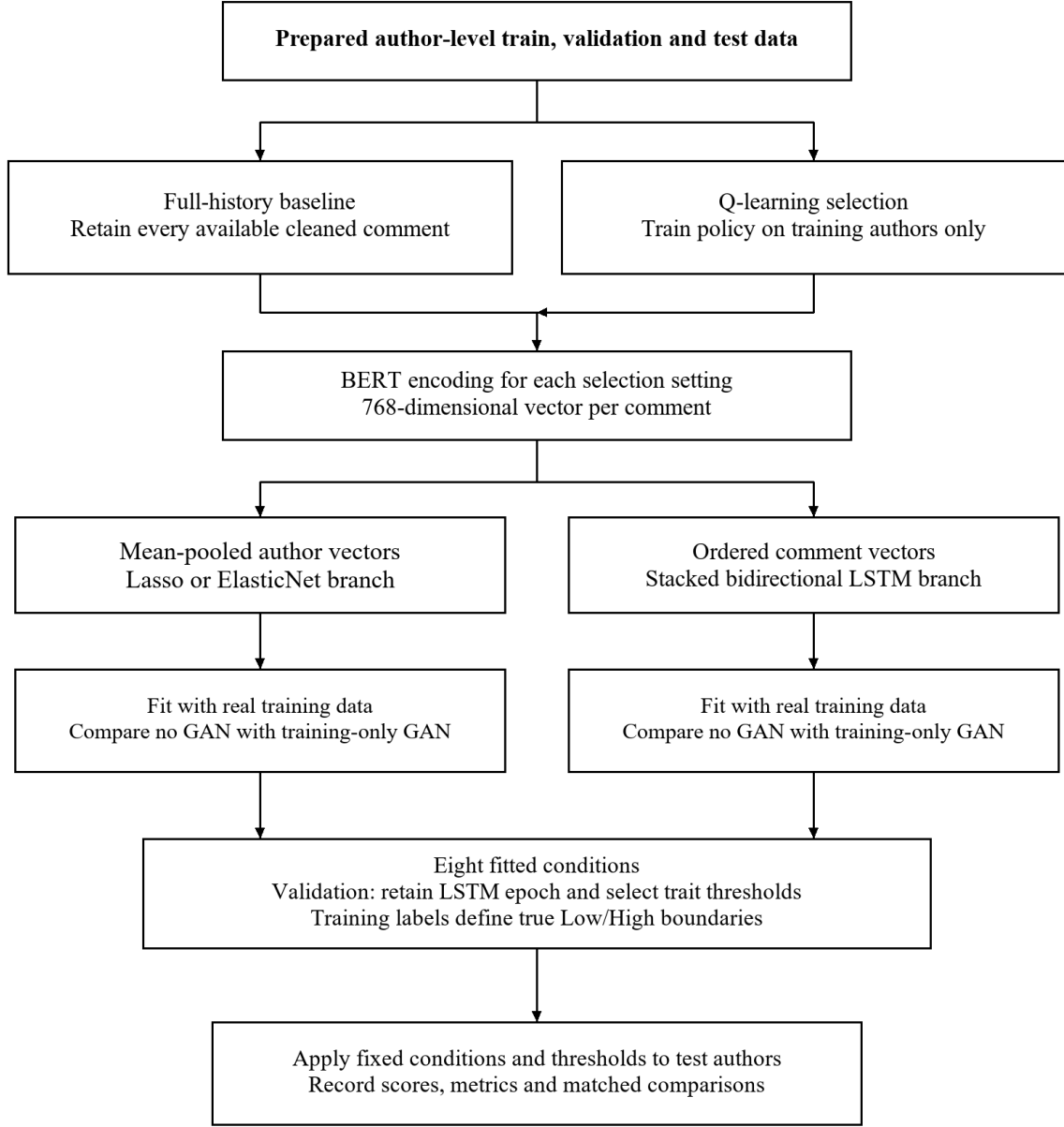
This separation is important when interpreting the report. The study does not claim that every PANDORA author registered through the application or completed a questionnaire administered by the researcher. Likewise, results obtained from the public demonstration or a small local volunteer collection are not substituted for the held-out results of the eight-condition experiment.

3.6 Personality Prediction Pipeline

The personality prediction pipeline converts prepared comments into five estimated trait scores. It follows a common selection and encoding process before separating into the pooled regression and sequence-model branches. Augmentation is applied only to the training representations of conditions that request it. Validation and test representations remain based on real comments.

The complete process is shown in Figure 3.3. The diagram distinguishes fitting operations from evaluation operations and shows the validation threshold step before final test reporting. This order is necessary because both the training procedure and the decision

rule contribute to the result. A test score is meaningful only when those choices have already been established.



Validation and test observations are never GAN training examples.

Figure 3.3: Processing Flow for the Eight-Condition Experiment

3.6.1 Baseline and Q-Learning Comment Selection

The baseline passes every available cleaned comment for a sampled author to the embedding stage. It provides a full-history reference against which the learned selector is compared. The baseline does not take only the first ten comments in the current experiment

runner. Its purpose is to show the predictive outcome and embedding workload associated with using the complete available text collection.

The Q-learning branch considers the same candidate comments and learns whether to select or skip each one. It uses a tabular action-value representation following the Q-learning framework described by Sutton and Barto (2018). The state describes the current comment together with the selection context. Its features include comment length, lexical diversity, similarity to already selected comments, the number of selected comments and the remaining candidates.

Lexical diversity is estimated from the proportion of distinct words in a comment. Redundancy is estimated by comparing simple term-frequency representations. These measures are inexpensive and can be computed before BERT encoding. Their use supports the objective of selecting informative and non-redundant comments without requiring a full transformer calculation for every learning step.

The reward for selecting a comment combines its informativeness and novelty and subtracts a small selection cost. Skipping gives no selection benefit. The agent therefore learns from a text-based proxy for usefulness. It does not receive a reward based directly on improved personality prediction accuracy. The effect of the learned selection on personality prediction is established later through the matched model comparisons.

During training, an epsilon-greedy rule allows the agent to explore alternative actions while generally using its current action values. The current runner initializes the learning rate at 0.1, the discount factor at 0.99 and the exploration probability at 0.1. The configured training epochs control the number of passes over training comment collections. Validation and test comments do not update the Q-table.

The selector supports a maximum comment budget, and its state includes the available selection capacity. However, the current full-history experiment calls it without a fixed top-k cap. Its upper bound is the number of available comments, so the policy may select a small subset or many comments. The presence of a top-k configuration field is not evidence that every current Q-learning condition is limited to ten comments.

During evaluation, selection uses the learned policy without exploratory learning updates. Selected comments remain in their retained order. If the policy returns no comment for an author, the experiment uses the first available comment as a fallback so that an

embedding can still be formed. This behaviour should be considered when examining selection counts, particularly for authors with short histories.

The usefulness of Q-learning is evaluated through both prediction quality and processing reduction. A reduction in selected comments establishes a reduction in the number of comment embeddings requested, but it does not automatically establish a faster total experiment or a more accurate model. Those claims require the corresponding timing and held-out performance evidence.

3.6.2 BERT Contextual Embedding Extraction

Selected comments are passed to the BERT encoder. The implementation uses bert-base-uncased and extracts a 768-dimensional vector from the final hidden-state representation of the classification token. BERT supplies contextual language representations that account for surrounding text (Devlin et al., 2019). The project uses these representations as inputs to the downstream prediction models.

The tokenizer prepares each cleaned comment using the configured maximum token length. The experiment default is 256 tokens per comment, and longer comments are truncated by the tokenizer. The resulting attention mask identifies valid input positions. Encoding is performed with the pretrained model in evaluation mode and without gradient updates, so the main experiment does not fine-tune BERT on personality labels.

Each comment produces one feature vector. For an author with several selected comments, the vectors are stored in the order in which the comments were retained. The regression representation is the mean of the vectors across that author. It therefore has 768 dimensions regardless of the number of comments. This provides a fixed-size input for the sparse regression models.

The LSTM representation retains the vectors as a sequence. If an author contributes m selected comments, the unpadded sequence contains m rows and 768 columns. This arrangement retains variation between comments that is removed by the initial mean-pooling operation. The two model branches therefore begin with the same type of comment representation but organize the evidence differently.

Embedding caching avoids repeating an encoding operation for the same cleaned text and token-length setting. Baseline and Q-learning conditions can reuse previously computed

vectors where their selected comments overlap. Cache reuse is an implementation efficiency and does not change which comments a selection method chose. The saved-call estimate for Q-learning is consequently interpreted separately from wall-clock savings due to caching.

3.6.3 GAN-Based Training Data Augmentation

The augmentation stage expands the training representations with generated embedding and personality-score pairs. The current implementation contains an adversarial generator and discriminator, consistent with the general GAN framework introduced by Goodfellow et al. (2014). It is therefore described as GAN-based augmentation rather than simple random perturbation of stored embeddings.

The generator receives a random latent vector and uses a shared internal representation with two output heads. One head produces an embedding and the other produces its five OCEAN values. The discriminator receives the embedding and score vector together and learns to distinguish generated pairs from real training pairs. This joint arrangement supplies the feature-target relationship required for supervised model fitting.

The process does not generate new comments, recruit additional participants or administer additional questionnaires. A synthetic embedding is a numerical training example. Its accompanying OCEAN values are generated targets rather than independently observed personality measurements. The generated observations therefore remain separate from the author counts used to describe the real dataset.

Before fitting the GAN, the experiment examines Low, Medium and High training-band counts for each trait. Authors belonging to underrepresented bands are identified as candidates for the augmentation pool. Where at least two suitable author positions are available, the GAN uses the focused pool. Where this is not possible, the implementation can use the wider training pool or skip generation when too little usable material remains.

The pooled regression branch fits the GAN to author-level mean embeddings and their OCEAN vectors. Generated pooled embeddings pass through the same feature transformation learned from real training observations. They are then added to the fitting matrix with their generated trait targets. Validation and test embeddings are not included in fitting the GAN or in estimating the feature scaler.

The sequence branch fits the GAN to training comment vectors, repeating the corresponding author's OCEAN vector across that author's training positions. Generated vectors are grouped into sequences with lengths taken from the selected real training sequences. The generated score vectors within each synthetic sequence are averaged to form its five target values. These sequences provide additional numerical training material, but the GAN does not explicitly learn a conversational or chronological sequence model.

Generated values are checked for invalid numerical entries and can be constrained to observed training ranges. OCEAN outputs are additionally constrained to the configured trait domain. The generator and discriminator losses and sample diagnostics provide information about the process. These checks help identify numerical problems, but they do not establish that a generated author representation has independent psychological validity.

Synthetic observations receive a lower fitting weight than real observations. The default synthetic weight is 0.35, while real-author weights are derived from training-band frequencies. The experiment records whether generation actually occurred and how many rows or sequences were added. A condition named as a GAN condition may fall back to unaugmented fitting if generation is skipped, and that outcome must remain visible in the report.

GAN effectiveness is assessed by comparing each augmented condition with its corresponding non-augmented condition. This comparison allows augmentation to help, have little effect or reduce performance. The methodology does not presume that adding more generated vectors will improve a model trained on a limited number of real authors.

3.6.4 Lasso and ElasticNet Personality Prediction

The regression branch fits a separate model for each OCEAN trait using the pooled author embeddings. Lasso uses an L1 penalty, while ElasticNet combines L1 and L2 penalties. These forms of regularization are suitable for examining high-dimensional features with a relatively modest number of authors (Tibshirani, 1996; Friedman et al., 2010).

The feature scaler is fitted once using real training-author vectors. It stores the means and scales needed to transform later inputs. The same transformation is then applied to validation, test and synthetic pooled vectors. This prevents the external evaluation data from contributing to the feature-normalization statistics.

For each trait, the model receives the transformed training matrix and the corresponding normalized target scores. Where augmentation is enabled and successful, generated rows and their generated targets are appended to this fitting data. Training sample weights are supplied so that the contribution of real and synthetic observations reflects the weighting procedure described earlier.

The trainer can choose regularization settings through internal cross-validation when the fitting sample is large enough. The current implementation uses up to five internal folds and a range of penalty values. With very small samples, it uses fixed settings. These internal folds are part of the regression fitting procedure; they do not replace the external validation and test partitions and do not turn the entire experiment into five-fold cross-validation.

The internal search is also not a fully nested evaluation of the complete pipeline. Preparation and augmentation occur before the regression search operates on its fitting matrix. The external test set remains the main source of final evidence. Internal scores are used to support fitting and are not reported as though every upstream stage had been independently repeated within each inner fold.

After fitting, the five regression models return a continuous trait-score vector for each evaluated author. Their coefficients can be inspected, including the number of non-zero feature coefficients. However, an embedding dimension does not directly represent a named behaviour or psychological cause. The model is relatively inspectable compared with the deeper sequence model, but its coefficients do not provide a complete natural-language explanation of a person's personality.

3.6.5 Stacked Bidirectional LSTM Personality Prediction

The LSTM branch processes the sequence of comment embeddings rather than their initial average. LSTM networks were introduced to support learning across sequences through a memory mechanism (Hochreiter & Schmidhuber, 1997). In this project, the sequence positions represent comments, while each position already contains a contextual BERT representation of its text.

The implemented model has two stacked bidirectional LSTM layers by default. Each direction uses 128 hidden units. Processing in both directions produces a 256-dimensional

output at each valid sequence position. The model can therefore consider relationships within the supplied comment order before producing a pooled author representation.

Authors have different numbers of comments, so sequences are padded for batch processing. Their true lengths are retained and supplied to the recurrent layer. Packing and a valid-position mask prevent padded positions from being counted as real comments during pooling. This is necessary because a shorter author history should not be represented as though it contained additional empty observations.

The recurrent outputs are averaged across valid positions, passed through layer normalization and dropout, and supplied to a regression head. The head contains a 64-unit hidden layer with a ReLU activation and a final layer with five outputs. There is no final softmax layer assigning Low, Medium or High categories. The model jointly predicts the five continuous trait values.

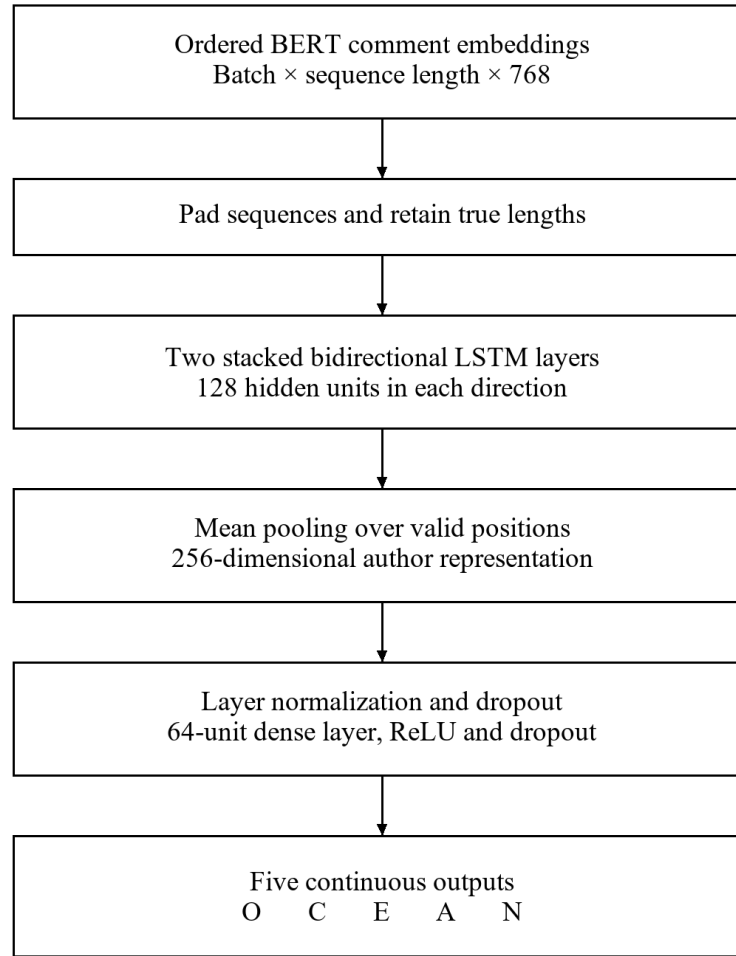


Figure 3.4: Stacked Bidirectional LSTM Architecture for OCEAN Prediction

The model uses SmoothL1 loss on the five targets. The loss is averaged across traits for each author and weighted using the author or synthetic-sample weight. Adam updates the model parameters, and gradient clipping limits unusually large gradient values. Dropout and weight decay provide additional regularization, although none of these settings guarantees that a small dataset will support strong sequence learning.

The default configuration uses 35 epochs, a batch size of four, dropout of 0.2 and a learning rate of 0.001. These are implementation defaults rather than claimed optimal values. The configuration saved with a run identifies the settings used in that experiment. They should not be replaced in the report by assumptions based on a different notebook or an earlier application version.

Validation predictions are monitored during training, and the weights from the epoch with the lowest validation MAE are restored. Validation therefore has two development roles in the LSTM branch: choosing the retained model weights and choosing the later binary decision thresholds. Test authors do not participate in either operation. Training loss histories remain diagnostic information rather than the final assessment of generalization.

Figure 3.4 distinguishes the recurrent model from the BERT encoder and from the later threshold stage. It also shows why the model should be described as a sequence regressor even though the service filename retains the word classifier. The implemented outputs and loss function determine its role in the methodology.

3.6.6 Trait Labels and Decision Thresholds

Continuous OCEAN prediction is the main modelling task. Binary evaluation is added to examine how the scores support a Low/High decision rule. The first threshold is the trait-label boundary, obtained as the median of real training labels for each trait. This boundary is fixed and used to label the true validation and test scores. It is a dataset-based division rather than a diagnostic personality threshold.

The second threshold is the model decision threshold. It is selected separately for each trait and experimental condition using validation prediction scores. Candidate values are obtained from the distribution of those scores. The metrics service evaluates the candidates and chooses the threshold that maximizes the harmonic mean of F1-score and specificity.

Combining F1-score and specificity gives attention to both High and Low cases. F1-score reflects precision and recall for High predictions, while specificity measures the correct identification of Low cases. The rule reduces the attraction of a decision threshold that gives an apparently favourable High-class score by assigning almost every author to High.

After selection, the validation threshold is fixed and applied to the test predictions. The test data are not used to choose a new official threshold. Additional threshold sweeps may be retained as sensitivity diagnostics, but a best-looking test threshold from such a graph is not substituted for the validation-selected operating point. This distinction is essential when presenting the threshold charts generated by the application.

3.6.7 Personality Profile Generation and Result Presentation

The predicted scores are consolidated into an author-level personality profile. A profile may contain the five observed scores, the five predicted scores, derived Low/High labels and a record of agreement for each trait. It is associated with the source experiment and model condition so that the prediction can be traced to its training context.

The experiment runner retains detailed prediction evidence for the held-out comparison. The researcher interface also supports a separate batch of inspectable test profiles. In the current interface, this secondary profile helper embeds up to the first ten cleaned comments and does not replay the learned Q-learning selection policy. Its profiles are therefore useful for inspection, but their aggregate measures are not substituted for the official eight-condition results.

This distinction is especially relevant when a Q-learning condition is chosen for a profile batch. Loading its fitted model alone does not reproduce its full input-selection procedure. A valid matched comparison requires the runner's corresponding selection, feature preparation and thresholds. The report consequently bases the factor comparisons on the recorded experiment outputs.

The public demonstration page is another separate function. It uses heuristic calculations rather than executing the trained PANDORA pipeline for every request. Its displayed output is not evidence of held-out model performance. The research interpretation remains based on the experimental conditions, their recorded data partitions and their saved prediction evidence.

3.7 System Modeling and Design

System modeling describes the movement of information through the platform and the organization of its persistent records. The functional architecture and processing diagrams show how a researcher request becomes a recorded experiment. The database design establishes how run settings, condition metrics, thresholds and prediction profiles remain connected after execution.

The design is centered on an experiment run rather than only on a volunteer profile. This reflects the shift from executing one pipeline for a selected participant to comparing several conditions over a common author sample. The earlier volunteer-related tables remain in the application, but the PANDORA records provide the principal storage structure for the experiment described in this chapter.

3.7.1 Data Flow Representation

At the context level, the researcher submits experiment settings and receives status information and results. The dataset files supply comments, author identifiers and personality labels. The personality prediction system processes these inputs and produces model artifacts, evaluation records and profile evidence. The main experimental context therefore depends on PANDORA files rather than a live request to the X platform.

At the functional level, the system is decomposed into data preparation, selection, encoding, augmentation, prediction and evaluation. Data preparation preserves partition membership. Selection determines which comments are represented. Encoding produces the numerical features. Augmentation extends training data where requested. The prediction branches fit their respective models and produce scores for the evaluation service.

At the more detailed level, prediction includes preparing pooled vectors or padded sequences, applying the fitted transformation, loading or fitting the appropriate model, obtaining five scores and storing the corresponding evidence. Threshold application follows score prediction. This decomposition explains the operations represented in Figure 3.3 without adding several diagrams that repeat the same processing stages.

3.7.2 Database Schema

The revised logical database structure is shown in Figure 3.5. A researcher may own several experiment runs. Each experiment run may have several condition results, threshold results and dataset allocations. A prediction run may refer to a source experiment and contains the individual test profiles generated during that batch.

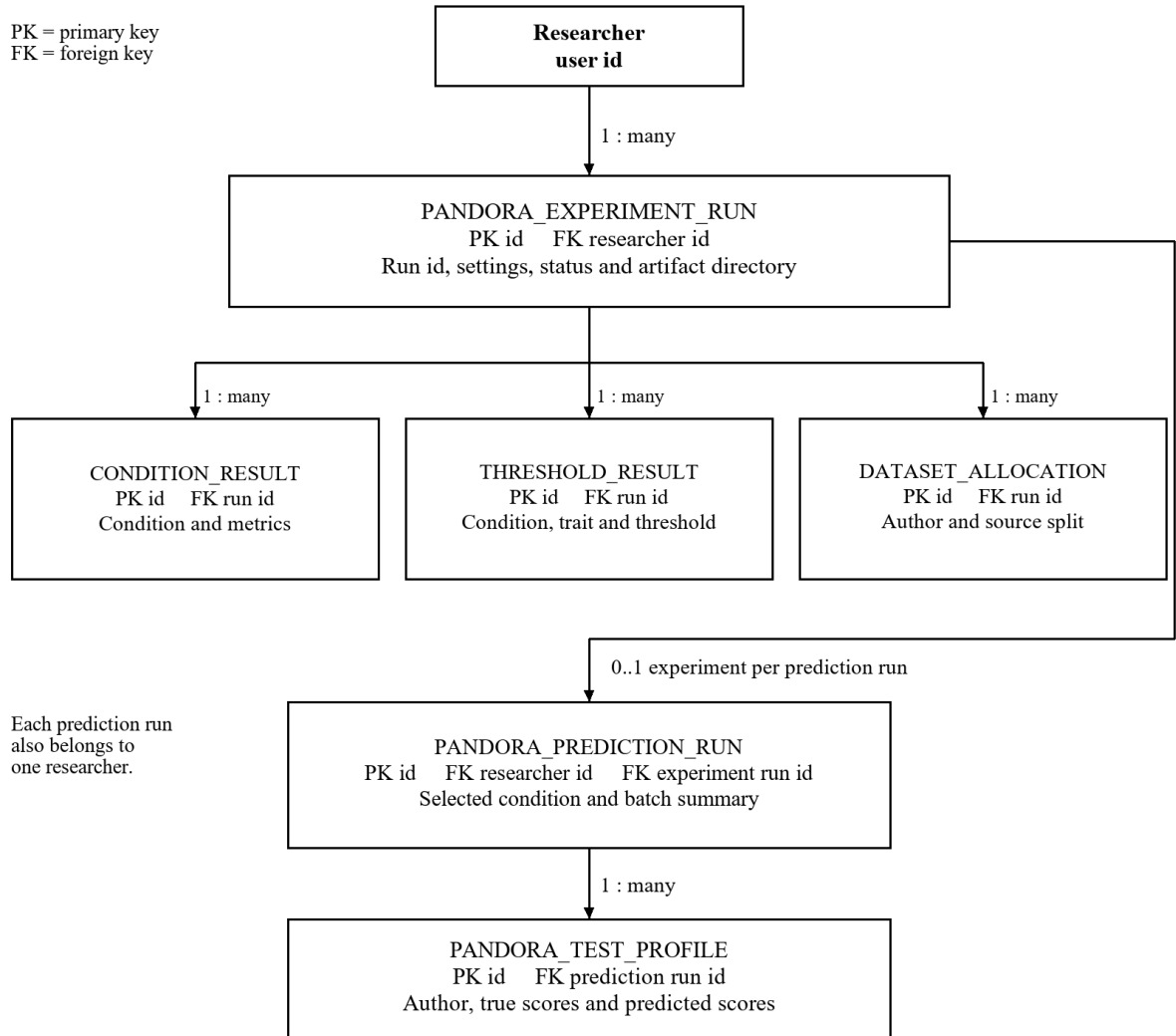


Figure 3.5: Entity Relationship Diagram for the PANDORA Research Records

The PANDORA_EXPERIMENT_RUN record stores the run identifier, researcher, status, dataset path, requested sample size, seed and artifact location. It also stores summary findings, audit status and error information. These fields provide the main reference for identifying a completed experiment or investigating one that did not complete.

PANDORA_CONDITION_RESULT stores a condition identifier with its model, selection and augmentation settings. It retains common summary measures and a detailed metrics field. A uniqueness rule combines the experiment run and condition identifier so that a run does not acquire two indistinguishable records for the same condition. The detailed metrics preserve information that cannot be represented by one summary accuracy field.

PANDORA_THRESHOLD_RESULT stores the split, trait, condition, candidate threshold and associated classification measures. Its direct database relationship is to the experiment run. The condition is stored as an identifying field rather than as a separate foreign key to the condition-result record. This allows the threshold rows to be grouped by run and condition for presentation.

PANDORA_DATASET_ALLOCATION connects a dataset author identifier and source split with the experiment run. It also includes fields for tracking the consumed record. These allocations support the inspection of author membership and reuse. Their existence assists the audit process, but the researcher must still distinguish legitimate reuse within matched comparisons from overlap between training and held-out authors.

PANDORA_PREDICTION_RUN stores a later prediction batch, its researcher, source file, selected condition, requested sample count and aggregate measures. Its link to the experiment is optional at database level. PANDORA_TEST_PROFILE stores the individual author identifier, comment count, observed and predicted OCEAN values, binary labels and correctness information for that batch.

This structure retains the distinction between an experiment, a condition and an individual prediction. The first identifies the shared research setting, the second identifies a combination of processing choices and the third identifies an author's output. Their separation makes the stored results easier to inspect without repeating the entire training procedure.

3.7.3 Researcher Interaction and Run Status

The researcher first selects the training operation and supplies the available settings. The application checks the form, creates a queued run and initiates processing. A run can then be reviewed through the experiment-history interface. The detail page presents the

condition comparison and the associated reporting information after the result has been stored.

If processing fails, the run records an error rather than a valid completed result. The error message helps identify problems such as missing dataset files, insufficient eligible authors or unavailable model dependencies. A failed run does not contribute prediction evidence merely because a database record was created for it.

Completed records allow researchers to examine earlier settings before starting another experiment. Where author reuse is permitted, it should be interpreted in the context of the experiment design. Repeating a comparison on the same test authors can support reproducibility checks, but repeatedly changing model settings in response to their scores weakens the independence of the final test claim.

3.8 Ethical Considerations and Data Protection

Personality prediction involves information about people and therefore requires care in data handling and interpretation. The main experiment uses existing dataset labels and text. It does not establish that the researcher obtained new direct consent from every PANDORA author. The use of these materials must remain consistent with the dataset's access conditions and the requirements applicable to the research setting.

The data required for the experimental pipeline consist of author identifiers, comments and OCEAN labels. Other demographic or personal information is not required by the described prediction inputs. Retaining only information needed for the study supports data minimization. Author identifiers remain useful for partition checks, but they should not be treated as proof that a dataset is completely anonymous.

Researcher authentication and ownership of run records provide basic application controls. Exported datasets, cached comments and saved artifacts also require controlled storage because they may contain source text or identifiers outside the database. Authentication alone does not establish comprehensive protection for every exported file or deployment environment. The chapter therefore describes the implemented controls without claiming a completed legal or security certification.

The supplementary volunteer workflow has a different consent context. Where the researcher collects questionnaire responses or retrieves posts for new volunteers, the purpose

and use of that collection should be explained to the participants. Those records must remain distinguishable from the existing dataset used for the main experimental comparison.

Generated samples are labelled and treated as synthetic training material. They are not presented as real participants or as new ground-truth observations. The final evaluation uses real held-out author records. This separation supports transparent reporting of the number of human observations underlying the performance measures.

Predicted traits are interpreted as estimates from available language, not as diagnoses or complete descriptions of a person. Platform conventions, topic, language variation and the limitations of the source labels may affect the result. The system is intended for research and educational use and does not establish suitability for recruitment screening, clinical decisions or other consequential automated decisions about individuals.

3.9 Model Evaluation Metrics

The evaluation examines continuous-score prediction, Low/High decisions and the contribution of the experimental factors. Measures are calculated for each trait and summarized across traits where appropriate. The evaluated split, number of authors and decision-threshold source accompany the results. This prevents a single value from being interpreted without the conditions that produced it.

All eight conditions use the same core metrics service. The comparison therefore applies consistent definitions to the regression and sequence outputs. The analysis gives particular attention to held-out prediction error, while binary measures provide an additional view of score-based decisions. Training diagnostics remain separate from the final test results.

3.9.1 Continuous Prediction Error

Mean Absolute Error measures the average absolute difference between observed and predicted trait scores. Because the targets are normalized, MAE is interpreted on that normalized scale. A smaller MAE indicates that the model predictions are closer to the supplied personality labels on average. The measure does not depend on whether an error is above or below the target.

Root Mean Square Error first squares the differences, averages them and takes the square root. It gives larger mistakes more influence than MAE while returning the result to the target scale. Reporting both measures helps identify whether a condition has moderate average error but occasionally produces much larger errors for some authors.

An error of 0.10 on the unit scale is a difference of ten points on a source scale converted by division by 100. This is an illustration of the scale conversion, not an observed experimental result. The same conversion convention must be used when presenting predicted scores and their errors; otherwise, a table may make one condition appear better simply because its values were reported on a different scale.

Per-trait errors are retained because model behaviour can differ across Openness, Conscientiousness, Extraversion, Agreeableness and Neuroticism. The mean across traits provides a compact summary, but it does not show whether a gain in one trait was accompanied by a loss in another. The detailed results therefore remain part of the evaluation evidence.

3.9.2 Goodness of Fit and Correlation

The coefficient of determination, R-squared, compares prediction error with variation around the mean observed score. A higher value indicates a better fit according to this comparison, while a negative value can occur when predictions perform worse than the mean-reference calculation. Negative values are retained and interpreted rather than removed to improve the appearance of a result.

Pearson correlation measures the direction and strength of a linear relationship between predicted and observed values. It can show whether higher observed scores tend to receive higher predictions. It does not measure absolute agreement. A prediction series may move in the correct direction but remain consistently too high or too low, so correlation is considered alongside MAE and RMSE.

Small samples or nearly constant values can make these measures unstable or undefined. The metrics service includes handling for some degenerate inputs, and the evaluator must examine author counts and score variation when interpreting the returned values. A numerical fallback is not evidence of meaningful psychological association.

3.9.3 Low and High Classification Measures

Binary measures are calculated after the training-derived trait labels and validation-selected model thresholds have been established. High is the positive class and Low is the negative class. Accuracy is the proportion of correctly labelled cases. It is reported because it is familiar, but it is not sufficient where the class distribution is uneven.

Precision is the proportion of predicted High cases that are truly High. Recall is the proportion of actual High cases that the model identifies. F1-score combines precision and recall through their harmonic mean. Specificity is the proportion of actual Low cases correctly identified. Together, these measures reveal errors that can be hidden by an apparently acceptable accuracy.

The confusion matrix records true positives, false positives, true negatives and false negatives. It provides a direct way to inspect a condition that predicts almost all authors as one class. In such a case, the matrix shows the imbalance in decisions even if a single summary measure appears favourable. Confusion counts are retained per trait rather than inferred from a chart alone.

ROC-AUC evaluates the ranking of High and Low cases using continuous scores across possible thresholds. PR-AUC is reported through average precision and focuses on the precision-recall relationship. These measures describe ranking performance, while the official accuracy and F1-score describe the fixed operating threshold. Neither AUC measure changes the rule selected on validation data.

The averaging method also requires attention. A mean of trait-level scores is not necessarily equal to a measure calculated after pooling every author-trait decision together. The experiment comparison and a separate profile-batch summary may therefore use different aggregations. Results are compared only when the split, threshold procedure and averaging definition agree.

3.9.4 Validation Procedure and Prevention of Data Leakage

The main evaluation uses the separate file-defined training, validation and test partitions. The training set supplies fitted parameters and training-derived statistics. The validation set supplies the retained LSTM epoch and decision thresholds. The test set

supplies the final evaluation. This replaces the former description of a complete five-fold validation procedure for the whole pipeline.

Data leakage is considered at each fitted stage. Q-learning receives only training comment collections during policy updates. The GAN receives only training feature-target pairs. The regression feature scaler is fitted on real training vectors. Trait-label medians are calculated from training scores. These choices prevent the external held-out observations from supplying information to those fitting operations.

The independence of authors must also be verified. Retaining three different file names is insufficient if the same author appears in more than one file. The experiment records author membership, and the research-contract audit checks for overlap. Any overlap affects the interpretation of generalization and must be resolved or disclosed before using the result as independent test evidence.

The saved test threshold sweeps require particular care. They can show the sensitivity of a fitted model to alternative operating points, but they are not a permitted source for choosing the official decision rule. Similarly, selecting a winning condition after reviewing test outcomes is a descriptive comparison of those outcomes. It is not a new unbiased estimate of how that selected winner will perform on another dataset.

Repeated experiments should retain their seeds, source files and settings. Where several seeds are evaluated, comparisons should be matched within each seed and their variation reported. The availability of repeated-run support does not establish that repeated experiments were actually completed. The results chapter must state the number of runs represented by its evidence.

3.9.5 Evaluation of Selection and Augmentation Effects

The Q-learning comparison pairs each learned-selection condition with the baseline condition having the same model and augmentation setting. Differences in MAE, accuracy and F1-score show whether the selected text supported the downstream prediction task. The comparison uses the same author allocation, so a change cannot be attributed simply to testing on a different group of people within that run.

Selection efficiency is measured through available and selected comment counts and the estimated number of BERT calls avoided. The estimate compares the selected count with

the full-history baseline count. Feature-construction times and condition processing times provide additional operational information. They are interpreted with cache reuse and hardware in mind because avoiding an embedding request is not identical to a measured reduction in total runtime.

The GAN comparison pairs each augmented condition with its non-augmented counterpart under the same model and selection settings. Lower error or better classification measures may support the usefulness of augmentation for that condition. The generated-sample count and targeting report are examined at the same time so that a condition with skipped augmentation is not credited with a GAN effect it did not test.

The model comparison pairs the sparse regression and LSTM branches under common selection and augmentation settings. This establishes whether sequence processing offers an advantage over pooled embeddings in the observed experiment. The LSTM is not assumed to be superior because it is deeper. Its additional training cost and the limited number of real authors remain relevant when assessing the result.

3.9.6 Verification and Reporting Records

The reporting procedure retains the condition comparison, trait-level metrics, prediction evidence and threshold results. It also retains data-quality summaries, trait-label thresholds, imbalance reports, targeted GAN records and Q-learning efficiency information. These records connect each performance claim with the data and processing choices needed to examine it.

Classification metrics can be recomputed from the saved prediction evidence and checked against the stored summaries. The audit also examines the presence of the eight conditions and the recorded threshold procedure. A failed audit identifies a problem requiring review. A passing calculation audit establishes agreement between those records; it does not establish that the model is accurate or that the dataset represents every intended population.

Functional verification covers the practical workflow as well as the numerical calculations. Relevant checks include rejecting unusable inputs, preserving author allocation, producing 768-dimensional features, handling variable sequence lengths, recording skipped augmentation and linking prediction profiles to their source runs. These are the behaviours that must be examined when the implemented system is tested.

The project contains tests and research-contract checks, but their presence is distinguished from evidence that every check has passed for a particular run. The results chapter should report the executed verification and the actual retained outputs. This methodology does not introduce new numerical findings or claim a completed training experiment merely from inspection of the application source.

3.10 Chapter Summary

This chapter has presented the methodology for the developed personality prediction platform and its controlled PANDORA experiment. The system prepares labelled comments at author level, extracts contextual BERT features and compares sparse regression with a stacked bidirectional LSTM. Baseline and Q-learning selection are evaluated with and without training-only GAN augmentation, producing eight experimental conditions.

The chapter has also described the separation of training, validation and test data, the distinction between continuous trait scores and Low/High decisions, and the storage of experiment evidence. These procedures address the objectives stated in Chapter One while retaining the modular design of the application. The subsequent chapter presents the implementation outcomes and experimental results using the evaluation procedure established here.

REFERENCES

- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (Vol. 1, pp. 4171–4186). Association for Computational Linguistics.
- Friedman, J., Hastie, T., & Tibshirani, R. (2010). Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, 33(1), 1–22.
- Gjurković, M., Karan, V. M., Vukojević, I., Bošnjak, M., & Snajder, J. (2021). PANDORA talks: Personality and demographics on Reddit. In *Proceedings of the Ninth International Workshop on Natural Language Processing for Social Media* (pp. 138–152). Association for Computational Linguistics.
<https://doi.org/10.18653/v1/2021.socialnlp-1.12>
- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). Generative adversarial nets. In *Advances in Neural Information Processing Systems* (Vol. 27, pp. 2672–2680).
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.
- Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1), 267–288.